Mega MoE Fused MegaKernel

Dispatch, two GEMMs, SwiGLU and combine in one kernel

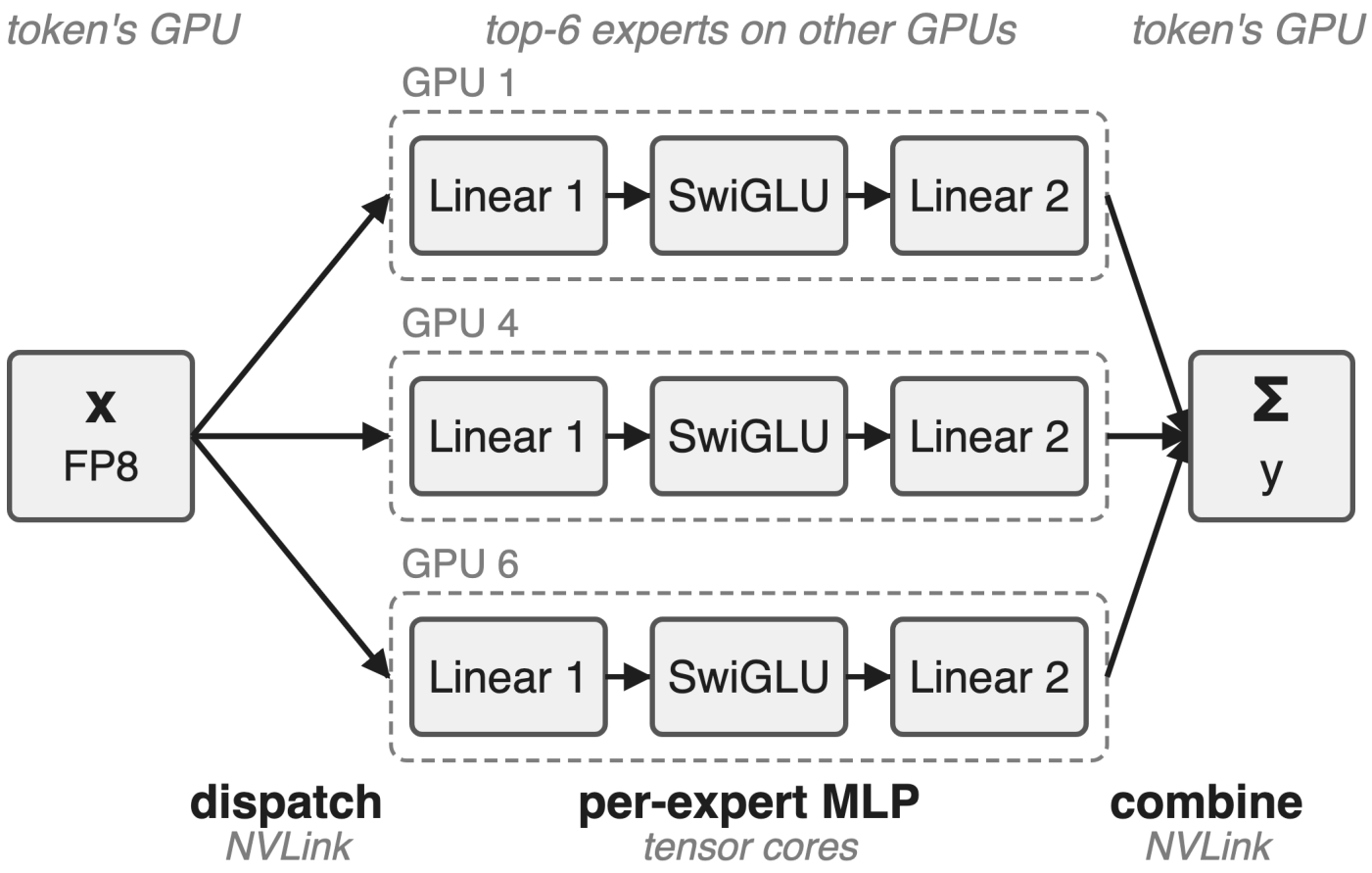
DeepSeek-AI · DeepGEMM Mega MoE (2026)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

GEMM & MoE Kernels · github.com/deepseek-ai/DeepGEMM

Each V4 MoE layer crosses GPUs twice

- V4-Flash: **256** experts, top-**6**
- V4-Pro: **384** experts, top-**6**
- EP over NVLink: tokens out, results back



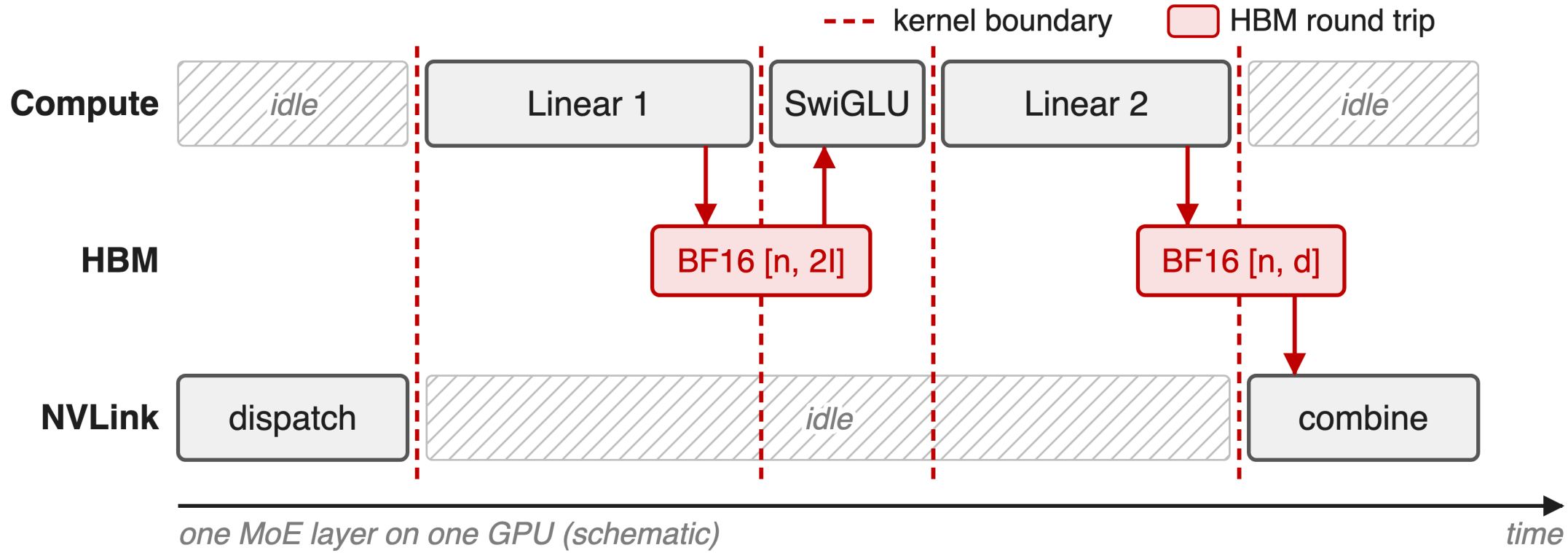
One token per rank still costs about 111 µs

≈ 111 µs

per V4-Flash MoE layer (routed experts, EP8) when dispatch, two GEMMs, SwiGLU and combine run as five kernels

Derived: 56.5 µs × 1.96 from DeepGEMM PR #316 (fused time × speedup vs the five-kernel "legacy" path)

Every kernel boundary is a layer-wide barrier



No expert starts before every token lands; intermediates round-trip HBM

Earlier overlap fixes fall short inside one batch

Batch-level overlap

- One batch's all-to-all hides under another's GEMMs
- **Needs a second batch**; RL-rollout tails have one

Per-stage overlap (Comet)

- Dispatch with linear 1, linear 2 with combine
- **1.42× ceiling** vs 1.92× with expert waves

Needed: overlap inside one batch, tile by tile, in one stream

Insight: NVLink all-to-all is just loads and stores

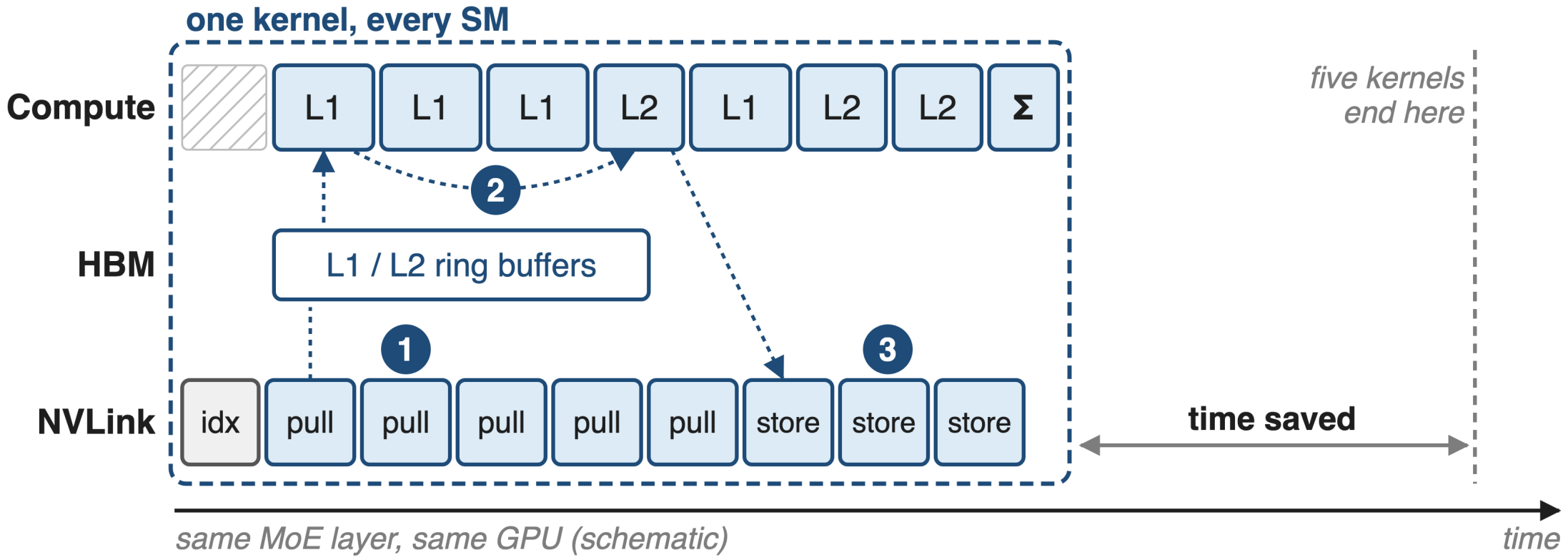
- Symmetric memory: peer address = own address + fixed offset
- V4 profiling: a MoE layer communicates less than it computes

Insight: NVLink all-to-all is just loads and stores

- Symmetric memory: peer address = own address + fixed offset
- V4 profiling: a MoE layer communicates less than it computes

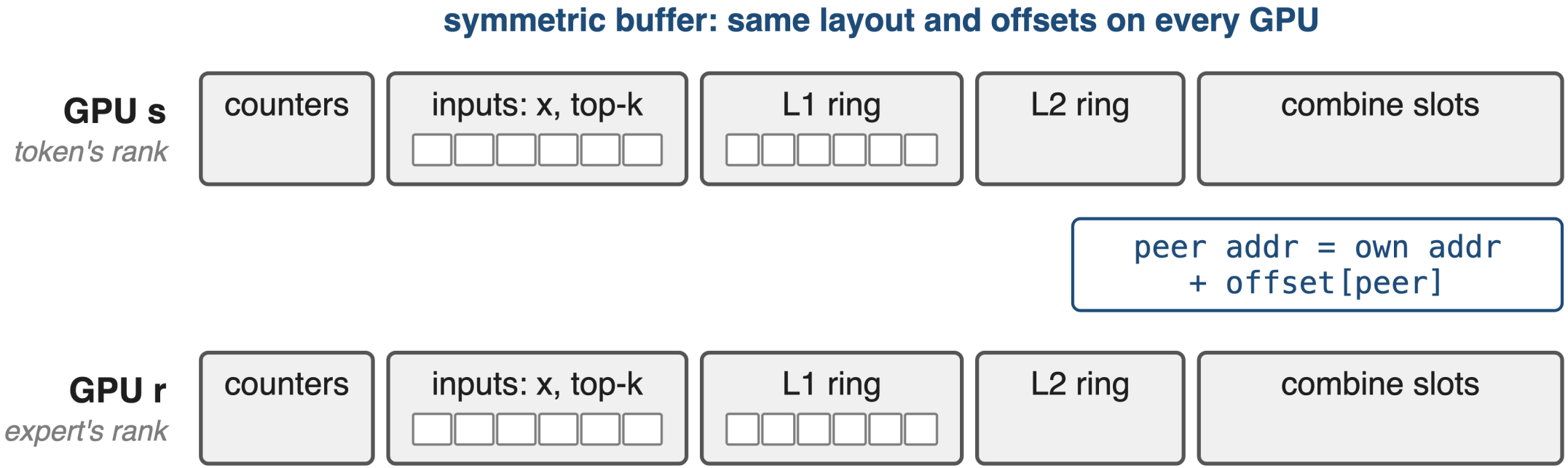
So GEMM-kernel warps can move tokens themselves, tile by tile, and compute stays the only bottleneck

One persistent kernel runs all five stages



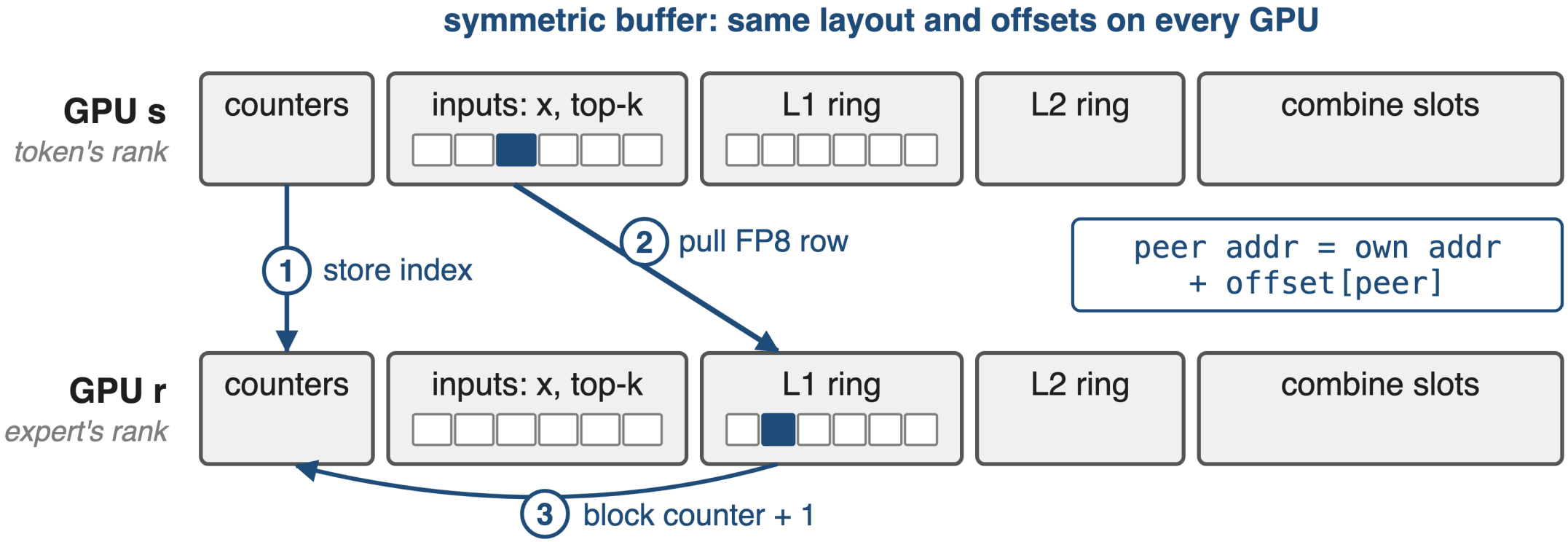
Schematic. ❶ pull dispatch · ❷ counter-gated linear-1/linear-2 tiles · ❸ epilogues: SwiGLU + FP8 cast, stores to peers, final Σ

❶ Dispatch warps pull tokens into expert tiles



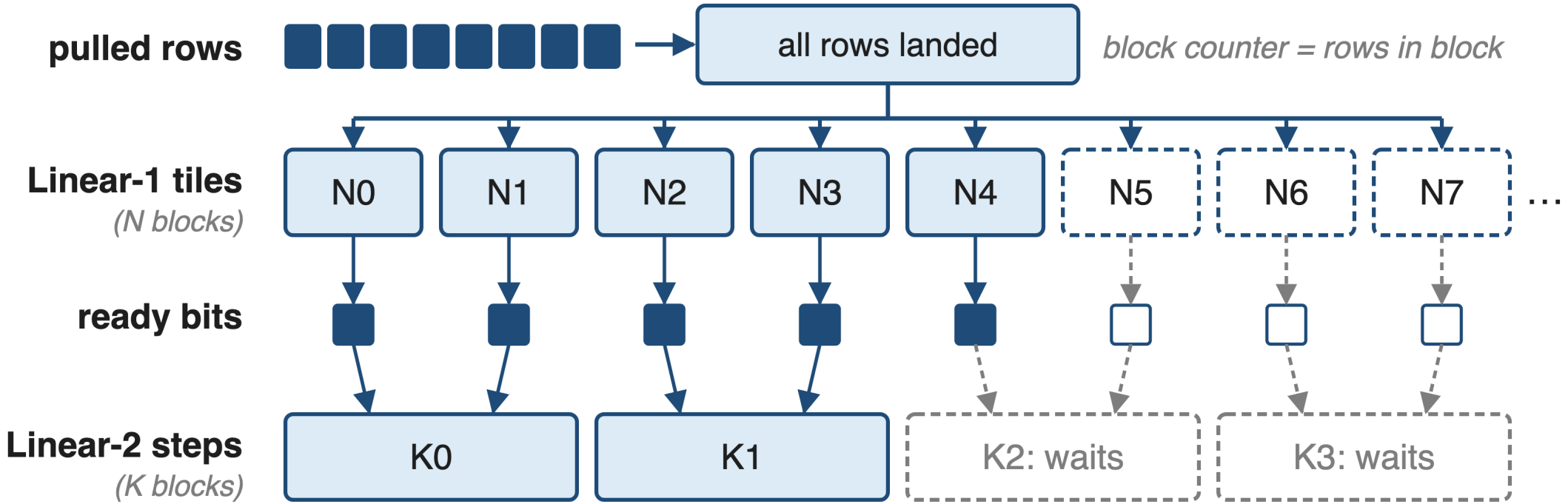
Pull, not push: the expert's GPU fetches each row it needs; no per-token notification

❶ Dispatch warps pull tokens into expert tiles



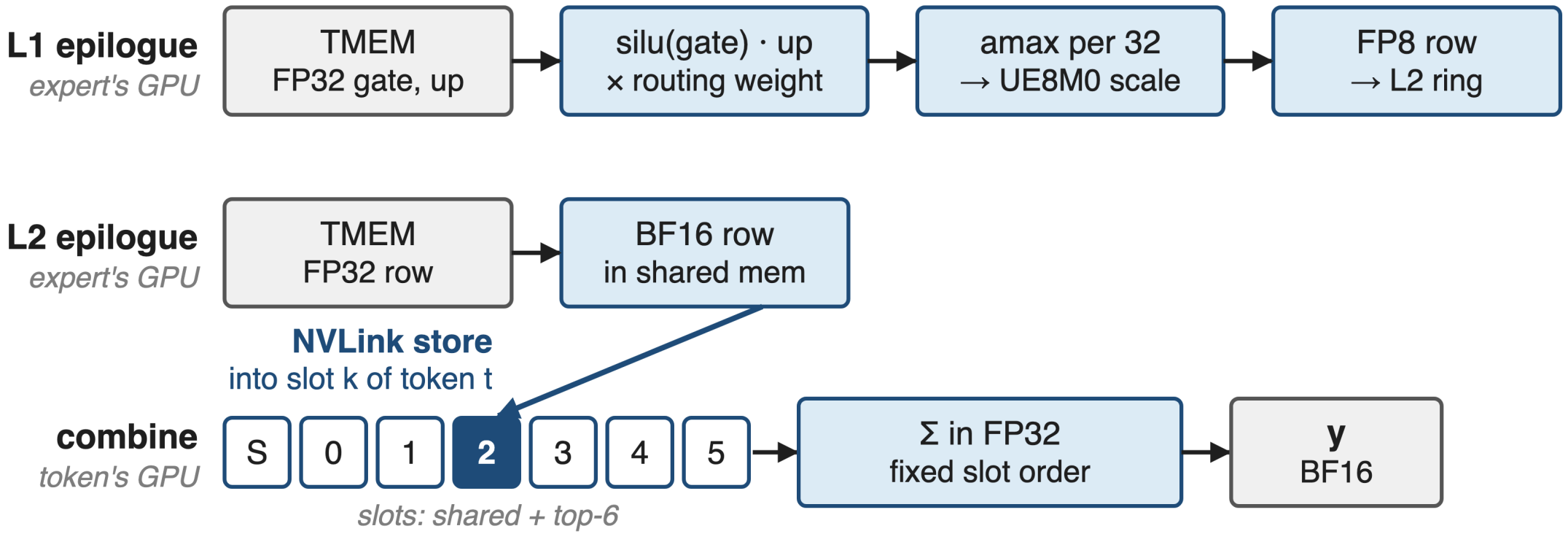
Pull, not push: the expert's GPU fetches each row it needs; no per-token notification

❷ Counters, not kernel ends, order the tiles



Linear 2 of a block starts while its last linear-1 tiles still run

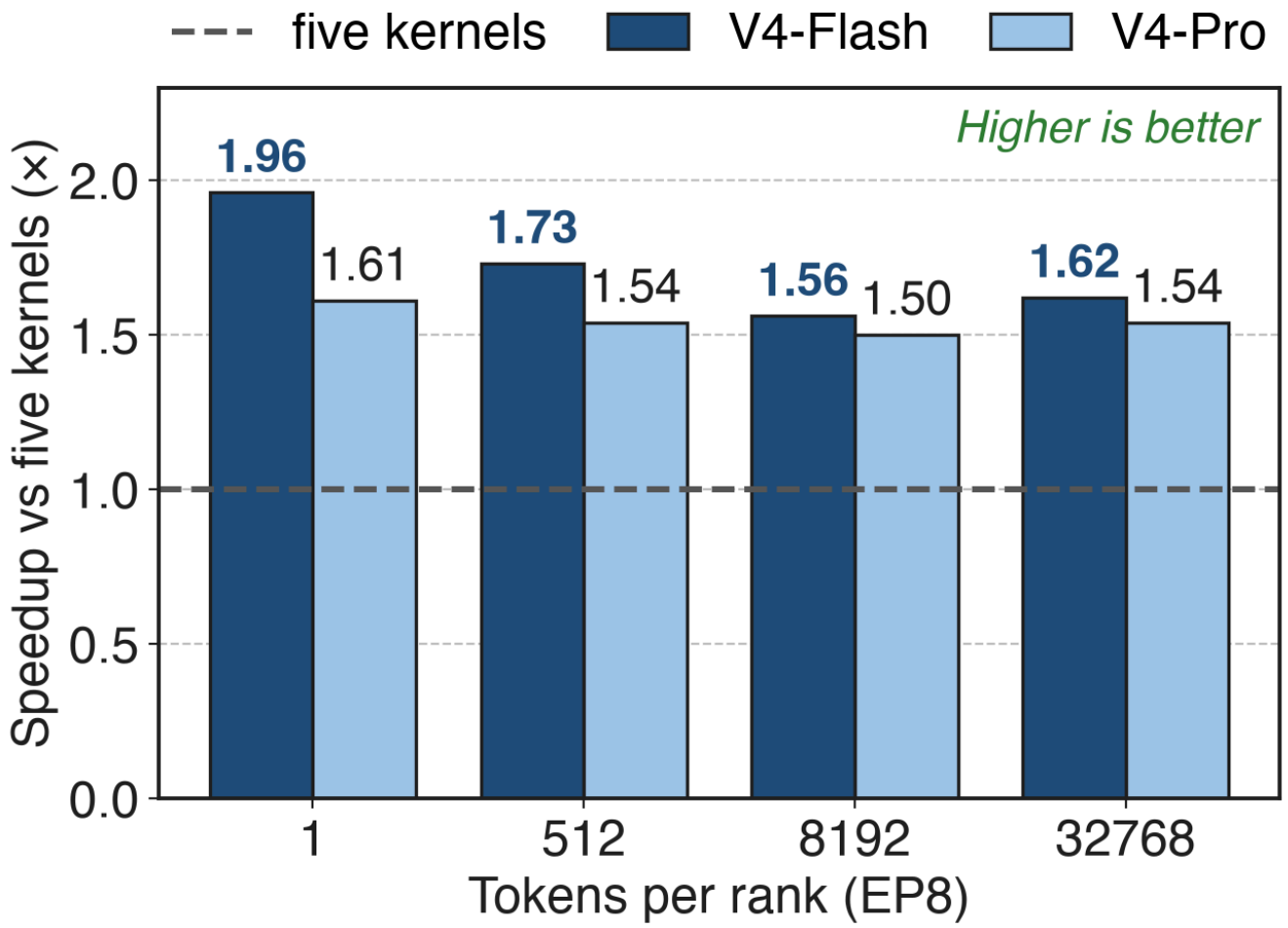
❸ Epilogues fuse SwiGLU, FP8 cast, combine



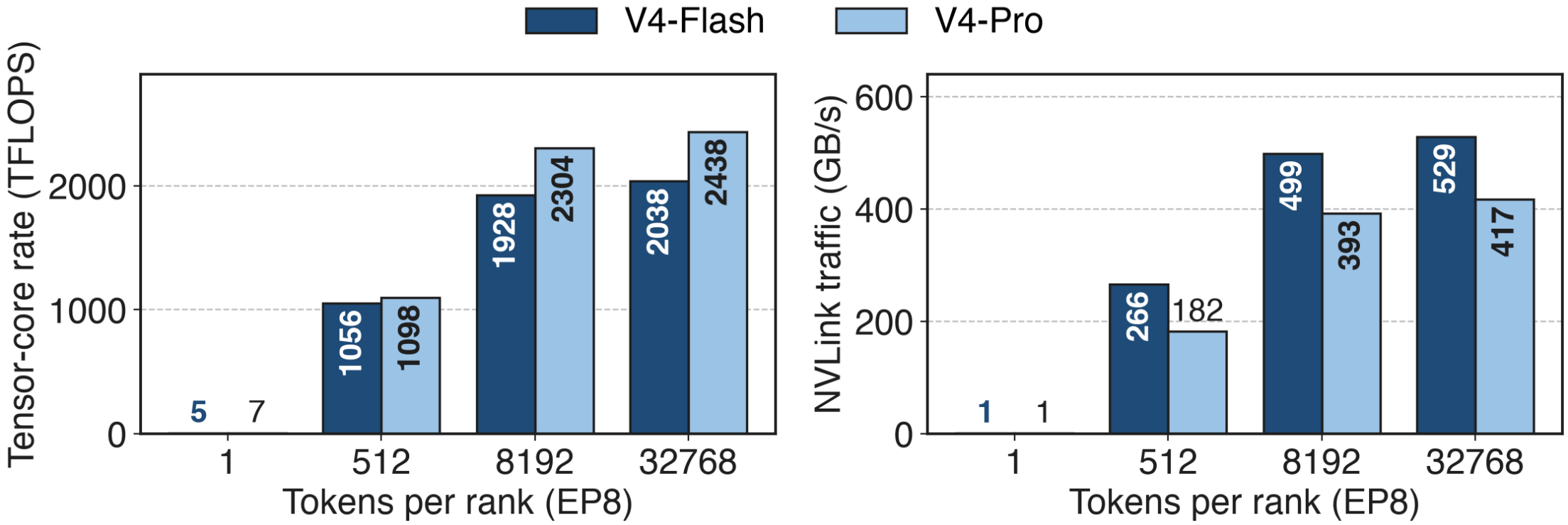
Linear-1 → linear-2 hand-off: 10·I → 2·I bytes per routed token (analytic)

One kernel runs 1.50–1.96× faster than five

- EP8 on SM100, both V4 models
- Biggest win at **1** token/rank: **1.96×**
- Baseline: DeepEP + DeepGEMM + TileLang, CUDA graph



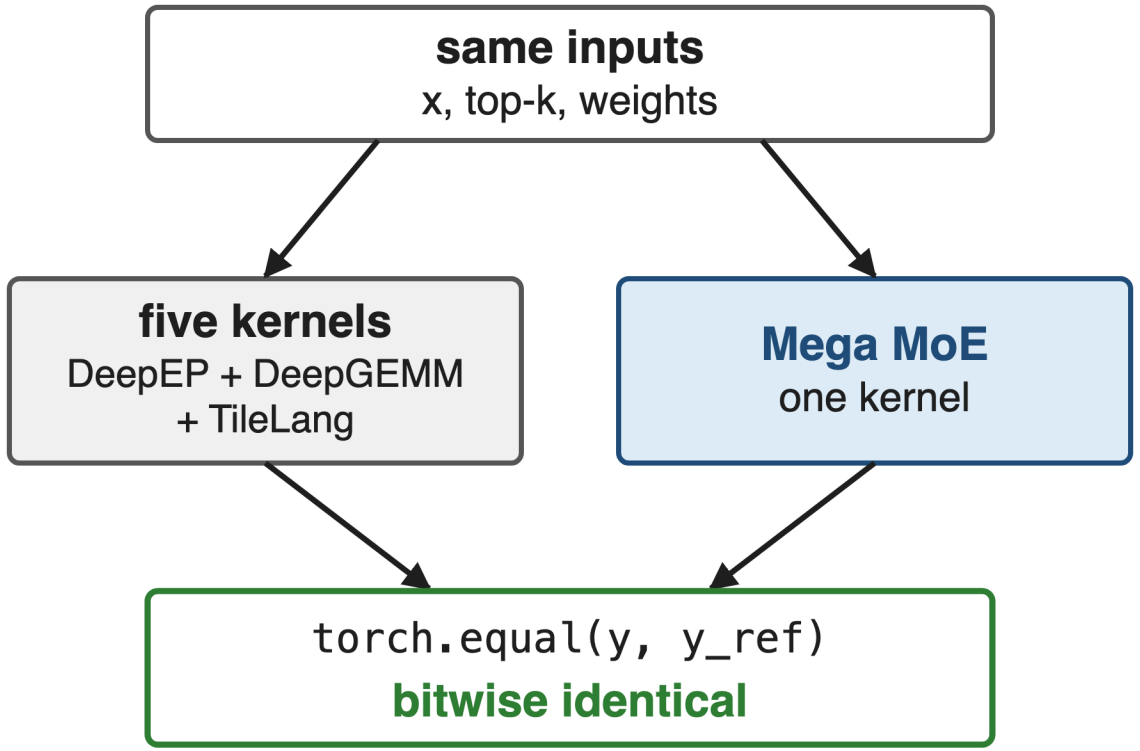
Big batches run 2+ PFLOPS while NVLink streams



V4-Flash, 32768 tokens: 2038 TFLOPS and 529 GB/s NVLink, both averaged over one 4.86 ms kernel

Fused output matches five kernels bit for bit

- Test asserts `torch.equal` on `y` and expert counts
- Combine sums shared + top-6 slots in FP32, fixed order



Determinism is opt-in since 2026-09: `use_deterministic_algorithms` (deck 39)

Later releases fused more and ran faster

- Jul: shared expert in, **11** kernels into **1**, **+20%**
- V4.1: **~15%** faster at low latency, **~4%** at high throughput
- Locality-aware: **100 W** saved, **5–10%** higher clock

Apr 2026 · PR #304
Mega MoE released, FP8 × FP4 only

Jun 2026 · PR #364
BF16 path, ring buffers cut memory

Jul 2026 · PR #377
shared expert fused, L1/L2 interleaved

Sep 2026 · PR #432
V4.1 tuning, determinism opt-in

Sep 30 2026 · PR #462
locality-aware scheduling

The cost: whole GPU, one NVLink domain, power

Prerequisites

- SM100 only; PyTorch ≥ **2.9** symmetric memory
- One NVLink domain (code caps at **72** ranks)
- SwiGLU only; occupies every SM

Costs

- Worst-case-sized buffer; inputs copied in first
- **Power**: compute, HBM and NVLink all run hot
- First pull and last sum stay exposed

Summary

- Five-kernel EP MoE: ≈ **111 µs** at 1 token/rank (V4-Flash)
- **Insight**: over NVLink, all-to-all is loads and stores
- Pull dispatch, counter-gated tiles, fused epilogues
- **56.5 µs**: **1.50–1.96×** faster, bitwise-equal output

https://github.com/deepseek-ai/DeepGEMM